

Modelos de Espacio de Estados (Mamba) para *Vetting* de Exoplanetas en TESS

José F. Zumbado Ruiz and Jeremmy Aguilar Villanueva

Escuela de Ingeniería en Computación, Instituto Tecnológico de Costa Rica, Cartago, Costa Rica

(Reporte Etapa 2, IC8104 Inteligencia Artificial, Semestre I 2026)

(Dated: 29 de mayo de 2026)



Resumen

Se evalúa empíricamente si un modelo selectivo de espacio de estados (Mamba) puede superar a baselines clásicos en la tarea de *vetting* de TESS Objects of Interest (TOIs) sobre curvas de luz de la misión TESS, bajo restricciones realistas de *dataset* y hardware. En términos simples, el objetivo es probar si Mamba puede distinguir mejor entre señales reales de planetas y falsos positivos usando la forma completa de la curva de luz, en lugar de depender solo de datos resumidos del catálogo. El conjunto experimental contiene 1 576 TICs etiquetados (CP / FP) con *splits* estratificados por TIC en proporción 70 / 15 / 15. Se entrena una escalera controlada de cuatro modelos bajo el mismo *split* y protocolo de evaluación: *random* estratificado, regresión logística sobre *features* del catálogo, una CNN *single-branch* inspirada en AstroNet, y Mamba sobre la vista global de la curva de luz. Sobre el *test* sellado, Mamba alcanza $AUC-ROC = 0,763$, superando al CNN por +15,9 puntos porcentuales bajo condiciones idénticas. Un *multi-seed sweep* de cinco semillas y su *ensemble* elevan la métrica a 0,806 con variabilidad estabilizada. El análisis de errores y la interpretabilidad multi-método (*gradient saliency*, *integrated gradients*, *occlusion*) coinciden en que los errores remanentes corresponden a curvas con SNR baja, no a modos sistemáticos del modelo. La reproducibilidad se garantiza con configuraciones YAML versionadas, semillas controladas, una suite de 49 pruebas automatizadas y un protocolo de *test* sellado evaluado una sola vez por modelo.

Índice

1. Introducción	4
1.1. Motivación	4
1.2. Objetivos de la Etapa 2	5
1.3. Contribuciones	6
2. Datos y preprocesamiento	7
2.1. Fuentes y construcción del dataset	7
2.2. Vista global	7
2.3. Data augmentation	8
3. Baselines	9
3.1. Random estratificado	10
3.2. Regresión logística sobre catálogo	10
3.3. CNN single-branch (AstroNet-inspired)	11
3.4. Mamba single-view	12
4. Protocolo experimental	14
4.1. Splits por TIC ID	14
4.2. Test sellado	15
4.3. Multi-seed como sustituto de K-fold	16
4.4. Prevención de <i>leakage</i>	16
5. Optimización de hiperparámetros	17
5.1. Sanity overfit por modelo	17
5.2. Configuración del modelo principal	18
5.3. Sweep multi-seed sobre la arquitectura <i>locked</i>	18
5.4. Curvas de aprendizaje y diagnóstico de <i>overfitting/underfitting</i>	19
5.5. Justificación de no usar <i>grid search</i> ni Optuna	21
5.6. Reproducibilidad por <i>run</i>	21
6. Métricas y resultados	21
6.1. Métricas utilizadas	21
6.2. Tabla principal	22
6.3. Curva ROC comparativa	23
6.4. Matriz de confusión y calibración del <i>ensemble</i>	24
7. Análisis de errores	25
7.1. Distribución de probabilidades por clase verdadera	25
7.2. Tasa de error por <i>feature</i> física	25
7.3. Casos más confidentes mal clasificados	26
8. Interpretabilidad: XAI	26
8.1. Métodos aplicados	27
8.2. Política de selección de casos	27
8.3. Hallazgos	27
9. Ingeniería y reproducibilidad	29
9.1. Estructura del repositorio	29
9.2. Configs YAML versionados	30
9.3. Semillas y determinismo	30
9.4. Artefactos por <i>run</i>	30

9.5. Entornos: Windows nativo vs WSL2	30
10.Suite de tests	31
10.1. Estrategia general	31
11.Discusión	31
12.Conclusiones	32

1. Introducción

1.1. Motivación

La misión TESS (*Transiting Exoplanet Survey Satellite*) de la NASA [5, 8] fue diseñada para observar a cadencia de 2 minutos las $\sim 200\,000$ estrellas enanas más brillantes del cielo, además de tomar imágenes de campo completo (FFI) que generan curvas adicionales para muchas otras estrellas. Cada sector observado dura ~ 27 días y produce series temporales del orden de $L \approx 18\,000$ puntos. De ese flujo de fotometría, una pequeña fracción muestra señales candidatas a tránsito planetario; la NASA cataloga estos candidatos como TOIs (*TESS Objects of Interest*).

La Fig. 1 ilustra la idea central del problema: una **curva de luz** registra el brillo de una estrella a lo largo del tiempo, y un posible tránsito aparece como una caída localizada en esa señal. La tarea de *vetting* consiste en distinguir esas caídas reales de las causadas por ruido instrumental, sistemas binarios u otros fenómenos astrofísicos que producen señales parecidas pero no corresponden a planetas.

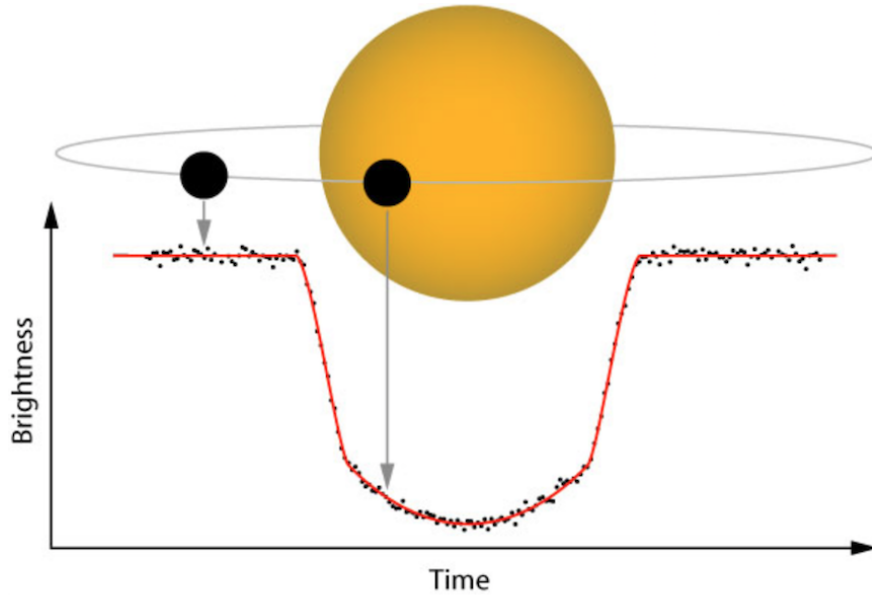


Figura 1: Representación conceptual de un tránsito planetario en una curva de luz. Cuando un planeta pasa frente a su estrella, bloquea una pequeña fracción de la luz observada y produce una caída temporal en el brillo medido. La tarea de *vetting* consiste en distinguir este tipo de señales reales de falsos positivos causados por ruido, estrellas binarias u otros fenómenos astrofísicos.

El catálogo TOI consultado para este trabajo registra más de 7 000 candidatos según el *snapshot* utilizado (al 1 de julio de 2025 se reportaban 7 655 TOIs, 638 con etiqueta CP y 1 987 con etiqueta FP). Por el costo de la revisión humana, el *vetting* de TOIs se apoya cada vez más en clasificadores automáticos.

Cada candidato del catálogo TOI tiene una **disposición**: una etiqueta breve que indica en qué punto del proceso de validación se encuentra. Algunas son provisionales (todavía hay que estudiar el candidato), otras son definitivas (ya se confirmó como planeta o se descartó). Para entrenar un clasificador supervisado el modelo necesita ejemplos con etiqueta definitiva: solo así puede aprender a distinguir lo que ya se sabe que es planeta de lo que ya se sabe que no lo es. La Tabla 1 resume las disposiciones del catálogo y cuáles se usan en este proyecto.

Cuadro 1: Disposiciones (`tfopwg_disp`) del catálogo TOI y su uso en este trabajo.

Disposición	Significado	Uso
CP	<i>Confirmed Planet</i> : planeta confirmado por TESS	Etiqueta positiva ($y = 1$)
FP	<i>False Positive</i> : señal descartada (binaria eclipsante, artefacto, etc.)	Etiqueta negativa ($y = 0$)
KP	<i>Known Planet</i> : planeta confirmado por misiones previas (Kepler, etc.)	Excluido por decisión experimental
PC	<i>Planet Candidate</i> : candidato aún sin confirmar	Excluido (sin etiqueta definitiva)
APC	<i>Ambiguous Planet Candidate</i> : candidato con evidencia ambigua	Excluido (sin etiqueta definitiva)
FA	<i>False Alarm</i> : artefacto instrumental	No considerado en este estudio

El *dataset* usado en este proyecto se restringe a TOIs con etiqueta **CP** (positivo) o **FP** (negativo). KP se excluye por decisión experimental: incluirlo mezclaría planetas conocidos por misiones anteriores (Kepler, K2) con confirmaciones obtenidas por la propia TESS, lo cual podría sesgar la evaluación. PC y APC se excluyen porque no tienen etiqueta definitiva todavía. Tras filtrar adicionalmente TICs sin cobertura fotométrica descargable mediante `lightcurve`, se obtiene el conjunto final de 1 576 TICs etiquetados.

Para situar este tamaño respecto a los baselines de la literatura, AstroNet [2] se entrenó sobre 15 737 TCEs etiquetados de Kepler (3 600 PC, 9 596 AFP y 2 541 NTP), y ExoMiner [3] reporta un *dataset* de trabajo de 30 609 ejemplos sobre $\sim 34\,000$ TCEs de Kepler DR25. El *dataset* de este proyecto es aproximadamente un orden de magnitud menor que el de AstroNet y cerca de 20 veces menor que el de ExoMiner. Esta diferencia de escala condiciona la magnitud absoluta de las métricas reportadas y se discute al evaluar la comparación con la literatura (Sec.11).

Los modelos de referencia en la literatura, como AstroNet [2] y ExoMiner [3], utilizan arquitecturas profundas basadas en representaciones de curvas de luz, con componentes convolucionales y vistas globales y locales. Este enfoque tiene dos limitaciones documentadas. Primero, el campo receptivo de una CNN crece de forma lineal con la profundidad y la captura de dependencias largas en secuencias de $L = 18\,000$ requiere arquitecturas profundas, costosas de entrenar. Segundo, las arquitecturas tipo Transformer, que sí modelan dependencias largas de forma nativa, tienen complejidad $\mathcal{O}(L^2)$ en memoria y cómputo, lo cual es prohibitivo bajo la restricción de hardware del proyecto (RTX 3050 con 4 GB de VRAM).

Los modelos de espacio de estados selectivos (Selective State Space Models, SSM) propuestos por Gu y Dao [1] ofrecen complejidad $\mathcal{O}(L)$ con memoria constante mediante un *selective scan*, y han igualado o superado a los Transformers en varias tareas de secuencia larga. Hasta donde se revisó en la literatura del proyecto, no se encontró una aplicación previa de Mamba al *vetting* de TOIs en TESS bajo restricciones de hardware similares. Esta brecha constituye la motivación técnica del proyecto.

1.2. Objetivos de la Etapa 2

La Etapa 2 del curso evalúa la fase de *Modelado, Entrenamiento, XAI y Evaluación*. Los criterios de la rúbrica fijan cinco ejes de trabajo. Este reporte se organiza para cubrir cada uno de forma explícita.

1. **Baselines obligatorios.** Entregar al menos dos baselines razonables y una comparación clara contra el modelo principal. En este trabajo se construye una *escalera* de cuatro

modelos (Sec.3) que permite atribuir cada incremento de desempeño a un componente concreto de la arquitectura.

2. **Validación y protocolo.** Adoptar un protocolo sólido con *test* ciego, *split* por TIC y prevención explícita de *leakage* (Sec.4).
3. **Optimización de hiperparámetros.** Realizar una búsqueda acotada y documentada de hiperparámetros, justificando explícitamente por qué no se ejecutó una búsqueda exhaustiva bajo las restricciones de VRAM y tiempo de entrenamiento (Sec.5). Por las mismas restricciones se sustituye la validación cruzada por una estrategia multi-semilla equivalente en términos de variabilidad estadística: con $K = 5$ y un *sweep* de ~ 20 configuraciones, la validación cruzada escalaría a más de 100 horas de GPU en la RTX 3050 de 4 GB de VRAM, lo cual es computacionalmente inviable en el contexto de este proyecto.
4. **Métricas y análisis de resultados.** Reportar métricas apropiadas a la clase desbalanceada, curvas ROC y PR, análisis de errores y variabilidad seed-by-seed (Secs. 6 y 7).
5. **Interpretabilidad (XAI).** Aplicar técnicas adecuadas al tipo de modelo y derivar hallazgos accionables (Sec.8). Dado que Mamba no implementa atención en el sentido del Transformer, se utilizan métodos basados en gradiente y oclusión en lugar de mapas de atención.

A esto se suma el eje transversal de **ingeniería y reproducibilidad** (Sec.9), cubierto mediante configuraciones YAML por experimento, semillas fijas, un protocolo de *test* sellado y una suite de pruebas automatizadas (Sec.10).

1.3. Contribuciones

Este reporte aporta lo siguiente.

1. Una **comparación controlada de cuatro modelos** sobre el mismo *split* sellado de TESS: *random* estratificado, regresión logística sobre el catálogo TOI, CNN *single-branch* inspirada en AstroNet [2], y Mamba con vista global como modelo principal.
2. Una **exploración experimental de Mamba aplicado al vetting de TOIs en TESS**. Hasta donde se revisó en la literatura del proyecto, no se encontró una aplicación previa bajo restricciones de hardware similares. El modelo se entrena en FP32 con *gradient clipping*, lo que permite procesar secuencias de longitud 18 000 dentro de 4 GB de VRAM.
3. Un **protocolo multi-semilla** (cinco semillas para Mamba *single-view*) que reemplaza la validación cruzada K -fold sin sacrificar la estimación de variabilidad. La sustitución se justifica en términos del costo estimado y se documenta en Sec.5.
4. Un **análisis XAI multi-método** (*gradient saliency*, *integrated gradients* y *occlusion sensitivity*) sobre Mamba, con una política de selección reproducible de ocho casos (los dos más confidentes por cuadrante TP, TN, FN, FP).
5. Una **infraestructura de reproducibilidad** que incluye configuraciones YAML versionadas, semillas controladas, una suite de 49 pruebas automatizadas, manifiestos de entorno por corrida y un protocolo de *test* sellado evaluado una sola vez por modelo.

El repositorio incluye además modelos exploratorios adicionales cuyos resultados están documentados en `paper/results/` pero quedan fuera del alcance del presente reporte.

2. Datos y preprocesamiento

2.1. Fuentes y construcción del dataset

El *dataset* se construye a partir de dos fuentes públicas de la NASA que se consultan programáticamente.

- **NASA Exoplanet Archive: TOI Catalog** [6]. Tabla pública con los TOIs (*TESS Objects of Interest*) y su disposición científica (CP, FP, PC, KP, APC, FA). Se descarga vía una consulta TAP (*Table Access Protocol*) ejecutada por `scripts/get_data.py`. Del catálogo se conservan únicamente las columnas relevantes para el experimento: identificador (`tid`), disposición (`tfopwg_disp`), y los parámetros de tránsito utilizados como *features* por la regresión logística (Sec.3.2): `pl_orbper`, `pl_trandurh`, `pl_trandep` y `st_tmag`.
- **MAST Archive** [7]. Almacén oficial de los archivos FITS producidos por la *pipeline* SPOC (*Science Processing Operations Center*) de TESS. Para cada TIC se descarga la curva PDCSAP_FLUX a cadencia de 2 minutos, que ya viene detrendada de sistemáticos instrumentales por la *pipeline* oficial. La descarga se realiza con la librería `lightkurve` [4], mediante el *script* `scripts/download_lightcurves.py`, idempotente y con *manifest* versionado (`data/splits/manifest.csv`).

La construcción del *dataset* sigue una secuencia de filtros declarada explícitamente:

1. Del catálogo TOI se conservan únicamente las disposiciones CP (*Confirmed Planet*, etiqueta positiva) y FP (*False Positive*, etiqueta negativa). Se excluye KP por decisión experimental (Sec.1.1); se excluyen PC y APC por carecer de etiqueta definitiva.
2. Se descartan TICs sin PDCSAP_FLUX accesible en MAST o cuya descarga vía `lightkurve` falla tras los reintentos configurados.
3. Se descartan TICs cuyo preprocesamiento global produce un tensor degenerado (por ejemplo, curva con más del 50 % de NaN o longitud útil menor a 5 000 puntos).

El conjunto resultante contiene 1 576 TICs etiquetados. Sus particiones *train*, *validation* y *test* se construyen por TIC (Sec.4) en proporción aproximada 70/15/15. Tabla 2 resume los conteos.

Cuadro 2: Conteos por *split* y clase. El *split* se realiza por TIC y de forma estratificada para preservar la proporción CP/FP entre particiones (Sec.4).

<i>Split</i>	N	CP	FP
Train	1 103	422	681
Validation	236	90	146
Test	237	91	146
Total	1 576	603	973

2.2. Vista global

La **vista global** es la representación principal del modelo Mamba y de las CNN *single-branch*. Se construye en `scripts/preprocess_global.py` con tres pasos:

1. **Lectura de la señal cruda.** Por cada TIC se concatenan los flujos PDCSAP_FLUX de todos los sectores descargados, ordenados por tiempo. Esta señal ya viene detrendada por SPOC y con artefactos sistemáticos removidos.

2. Normalización por curva individual. Sea $\mathbf{f} = (f_1, \dots, f_T)$ la señal cruda concatenada y $\tilde{f} = \text{median}(\mathbf{f})$ su mediana. La curva normalizada $\mathbf{x}$ se define como

$$x_t = \frac{f_t}{\tilde{f}}, \quad t = 1, \dots, T. \quad (1)$$

Intuitivamente, esta normalización hace que todas las curvas queden centradas alrededor de 1,0. Así, el modelo no aprende simplemente que una estrella es más brillante que otra (información irrelevante para detectar tránsitos), sino que se enfoca en la **forma relativa** de las caídas de brillo. La normalización se hace siempre con estadísticas de la propia curva, nunca con estadísticas globales del *dataset*: esta decisión evita el *leakage* de escala desde el *train* al *test* (Sec.4).

3. Ajuste a longitud fija $L = 18\,000$. Las curvas resultantes tienen longitudes variables (función del número de sectores observados). Para que el modelo opere con tensores de forma homogénea se aplica:

- Si $T \geq L$, se recorta al primer bloque contiguo de L puntos válidos.
- Si $T < L$, se rellena con la mediana de la curva (es decir, con el valor neutro 1,0 tras la normalización).

Los valores NaN restantes se interpolan linealmente sobre el índice temporal. La salida es un tensor `float32` de forma $(1, 18\,000)$ guardado como `data/processed/global/<tic>.pt`. Este contrato lo verifica `test_dataset.py` (Sec.10).

Qué modelos la consumen. La vista global es el *input* de los modelos de aprendizaje profundo evaluados en este reporte: la CNN *single-branch* (Sec.3.3) y Mamba *single-view* (Sec.3.4). No se construyen vistas adicionales sobre la curva.

2.3. Data augmentation

Para mitigar el tamaño reducido del *dataset* (Sec.1.1) se aplican cuatro transformaciones estocásticas a la vista global, definidas en `src/exoplanet/data/augment.py`. Estas se aplican **únicamente al *split* de *train***; *val* y *test* se evalúan con la señal cruda preprocesada, para que las métricas reflejen desempeño sobre datos no modificados.

Cuadro 3: Transformaciones de *augmentation* aplicadas al *split* de *train*.

Transformación	Operación	Hiperparámetros del proyecto
<code>temporal_shift</code>	Desplazamiento circular de la curva: $x'_t = x_{(t+s) \bmod L}$ con $s \sim \mathcal{U}\{-S, \dots, S\}$.	$S = 500$ (puntos).
<code>gaussian_noise</code>	$x'_t = x_t + \varepsilon_t$, $\varepsilon_t \sim \mathcal{N}(0, \sigma^2)$.	$\sigma = 0,001$ (orden del ruido instrumental TESS típico).
<code>time_reverse</code>	Inversión temporal $x'_t = x_{L-t+1}$ con probabilidad p . Válida por la simetría temporal del tránsito.	$p = 0,5$.
<code>amplitude_scale</code>	$x'_t = a \cdot x_t$, $a \sim \mathcal{U}(a_{\min}, a_{\max})$. Simula diferencias de calibración.	$a \in [0,99, 1,01]$.

temporal_shift: variabilidad en la posición temporal del tránsito. Una curva de luz no siempre tiene el tránsito en el mismo lugar dentro de la ventana de 18 000 puntos: depende del momento exacto en que empezó la observación del sector. Si el modelo solo viera tránsitos “en el centro” de la curva, podría aprender a buscar señal solo en esa región. El desplazamiento circular mueve la curva a izquierda o derecha por una cantidad aleatoria, generando ejemplos donde el mismo tránsito aparece en posiciones distintas. Esto fuerza al modelo a volverse **invariante a la posición temporal** de la señal, una propiedad deseable porque el momento del tránsito no debería influir en la decisión de si es planeta o no.

gaussian_noise: tolerancia al ruido fotométrico. Las curvas TESS reales contienen ruido instrumental y fotométrico: fluctuaciones asociadas al detector, a la estadística de fotones, a artefactos de procesamiento y a variabilidad estelar no relacionada con el tránsito. Sumar ruido gaussiano artificial de magnitud similar al ruido real ($\sigma = 10^{-3}$ es el orden de magnitud típico tras la normalización por mediana) entrena al modelo para que su decisión **no dependa de las fluctuaciones puntuales** sino de la estructura subyacente de la curva. Sin este *augmentation*, un modelo podría sobreajustarse al patrón exacto de ruido de las curvas de entrenamiento.

time_reverse: aprovechar la simetría del tránsito. La forma de un tránsito planetario es aproximadamente simétrica en el tiempo: el flujo decae cuando el planeta entra al disco estelar (ingreso) y se recupera cuando sale (egreso) siguiendo una curva espejo. Esto significa que si invertimos el tiempo de la curva, un tránsito real sigue pareciendo un tránsito real. La transformación explota esta propiedad para **duplicar efectivamente el número de tránsitos disponibles** sin necesidad de descargar nuevas curvas. La probabilidad $p = 0,5$ aplica la inversión a la mitad de los ejemplos por época, manteniendo diversidad sin sesgar el *dataset*.

amplitude_scale: tolerancia a diferencias de calibración. Aunque la normalización por mediana (Sec.2.2) centra las curvas alrededor de 1,0, pueden persistir pequeñas diferencias de calibración entre estrellas o sectores. Multiplicar toda la curva por un factor aleatorio cercano a 1 (en $[0,99, 1,01]$, es decir, hasta un 1 % de cambio) simula esas diferencias y entrena al modelo para que sea **robusto a desplazamientos multiplicativos pequeños en la escala absoluta**, una propiedad que en el límite ideal debería ser invariante en una tarea de clasificación de forma del tránsito.

Contrato funcional común. Las cuatro transformaciones cumplen el mismo contrato funcional: son puras (no mutan el tensor de entrada), preservan forma y `dtype`, son determinísticas bajo `torch.Generator`, y manejan correctamente los casos identidad (*e.g.*, $\sigma = 0$ no agrega ruido y devuelve un clon del tensor). La validación de este contrato se hace mediante 21 pruebas unitarias en `tests/test_augment.py` (Sec.10). El orden de aplicación y los hiperparámetros se declaran por experimento en el campo `augmentation` del YAML correspondiente.

3. Baselines

Para evaluar el aporte real de la arquitectura Mamba en este problema se construye una **escalera de cuatro modelos** ordenada por expresividad creciente. La idea detrás de la escalera es que cada modelo agrega exactamente un componente respecto al anterior, de modo que la diferencia de desempeño entre dos vecinos puede atribuirse a ese componente concreto y no a una combinación de cambios. La Tabla 4 resume los cuatro modelos y su rol.

Cuadro 4: Escalera de modelos. La columna *Aporta* describe el componente que cada modelo agrega respecto al anterior; el orden permite atribuir ganancias o pérdidas a una variable a la vez.

Modelo	Aporta respecto al anterior	Rol en el reporte
Random estratificado	Solo el <i>prior</i> de clase	Piso mínimo (Sec.3.1)
Regresión logística (catálogo)	Features tabulares del TOI	Baseline no-DL fuerte (Sec.3.2)
CNN single-branch	Aprende representación 1D de la vista global	Baseline DL clásico (Sec.3.3)
Mamba single-view	Sustituye convolución por SSM selectivo	Modelo principal de Etapa 2 (Sec.3.4)

El modelo entregable principal de Etapa 2 es Mamba *single-view*. Los tres modelos anteriores establecen una jerarquía creciente de expresividad contra la cual se mide su aporte real.

3.1. Random estratificado

El baseline más simple no mira la curva: solo conoce la proporción de clases en el *split* de entrenamiento. Para cada muestra emite como probabilidad constante el *prior* positivo de *train*:

$$\hat{p}(y = 1) = \frac{N_{\text{CP}}^{\text{train}}}{N_{\text{CP}}^{\text{train}} + N_{\text{FP}}^{\text{train}}} = \frac{422}{1103} \approx 0,383. \quad (2)$$

Como esta probabilidad queda por debajo del umbral de decisión 0,5, el baseline clasifica todas las muestras del *test* como negativas al calcular F1, *recall* y *precision*. Por eso su *recall* para CP es 0 (no recupera ningún planeta), aunque su AUC-ROC esperado permanece en 0,5 (la métrica no depende del umbral, sino del ordenamiento de las probabilidades, que es arbitrario cuando todas son iguales).

Para qué sirve. Establece el *piso* contra el cual todos los modelos posteriores deben superar. Si un modelo basado en la curva no supera este AUC de 0,5, es evidencia de un error de implementación o de *pipeline*, no de un problema de aprendizaje.

3.2. Regresión logística sobre catálogo

El segundo baseline aprovecha la información tabular ya presente en el catálogo TOI sin tocar las curvas de luz. Por cada TIC se construye un vector de *features* $\mathbf{x} \in \mathbb{R}^d$ con cuatro componentes tomadas del catálogo: el período orbital (`p1_orbper`), la duración del tránsito (`p1_trandurh`), la profundidad del tránsito (`p1_trandep`) y la magnitud TESS de la estrella anfitriona (`st_tmag`). Sobre estas se ajusta un modelo de regresión logística:

$$\hat{p}(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b), \quad (3)$$

donde $\sigma(z) = 1/(1 + e^{-z})$ es la función sigmoide. La función σ comprime cualquier valor real a un número entre 0 y 1, interpretable como probabilidad.

Intuición. La regresión logística funciona como una **regla de puntuación**: combina los datos del catálogo (período, duración, profundidad, magnitud), produce un único número real $\mathbf{w}^\top \mathbf{x} + b$ asignando un peso a cada *feature*, y luego transforma ese número en una probabilidad entre 0 y 1. Si la probabilidad es alta, el modelo considera más probable que el objeto sea un planeta confirmado; si es baja, lo clasifica como falso positivo. Los pesos $\mathbf{w}$ son exactamente lo que el modelo aprende durante el entrenamiento: en qué medida cada *feature* aporta evidencia a favor o en contra del tránsito.

Los parámetros $\mathbf{w}, b$ se aprenden minimizando la pérdida de entropía cruzada binaria con pesos por clase para corregir el desbalance:

$$\mathcal{L}_{\text{BCE}}(\mathbf{w}, b) = -\frac{1}{N} \sum_{i=1}^N \left[w_+ y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i) \right], \quad w_+ = \frac{N_{\text{FP}}}{N_{\text{CP}}}. \quad (4)$$

Decisión de diseño. Las *features* usadas vienen **directamente del catálogo**, no se recalculan desde la curva (por ejemplo, no corremos un algoritmo BLS propio para reestimar el período). Esta elección se documenta debido a que recalcular BLS sobre 1 576 curvas con grilla densa toma horas y el catálogo TOI ya tiene esos números con la calidad oficial de la NASA.

Para qué sirve. Establece cuánto desempeño se puede extraer de la metadata sin tocar la curva. Si un modelo basado en la curva no supera a la regresión logística, significa que la curva no aporta señal adicional sobre lo que ya está en el catálogo, lo cual sería un hallazgo relevante en sí mismo.

3.3. CNN single-branch (AstroNet-inspired)

Primera arquitectura de *deep learning* de la escalera. Se inspira en AstroNet (Shallue & Vanderburg 2018) pero opera sobre una sola rama (la vista global), lo cual la hace comparable directamente con Mamba *single-view*.

Operación básica de una capa convolucional 1D. Una capa convolucional aplica un filtro $\mathbf{k}$ de tamaño fijo K a la curva $\mathbf{x}$, deslizándolo a lo largo del tiempo:

$$y_t = \sum_{i=0}^{K-1} k_i \cdot x_{t-i} + b. \quad (5)$$

El filtro tiene los mismos parámetros para todas las posiciones temporales (*weight sharing*), lo que le permite detectar el mismo patrón local (por ejemplo, un *dip* de tránsito) sin importar dónde aparezca en la curva. Apilar varias capas convolucionales con *max-pooling* entre ellas agranda el *campo receptivo*: la profundidad D de capas con *kernel* K y *pooling* P produce un campo receptivo del orden de $K \cdot P^D$ puntos.

Arquitectura usada. La red recibe la vista global de la curva (un vector de longitud $L = 18\,000$) y produce un único número: la probabilidad de que la curva contenga un tránsito planetario real. Internamente la procesa en dos fases.

Fase 1: extracción de patrones locales. Se aplican cuatro **bloques convolucionales** en cascada. Cada bloque transforma la representación de la curva en una versión más abstracta y más corta:

- Una **convolución 1D** (Sec.5) recorre la curva con un filtro de tamaño 5 y produce una nueva señal. Si el bloque anterior produjo C señales paralelas (llamadas *canales*), el siguiente bloque las combina y produce C' canales nuevos. Los canales se pueden pensar como “detectores” especializados en distintos patrones (uno aprende a detectar *dips*, otro

detecta pendientes ascendentes, otras variaciones suaves de fondo, etc.). Estos detectores **no se programan manualmente**: la red los aprende ajustando sus pesos durante el entrenamiento, a partir de los ejemplos etiquetados del *split* de *train*. El número de canales crece con la profundidad: $1 \rightarrow 16 \rightarrow 32 \rightarrow 64 \rightarrow 128$. Mayor número de canales significa más patrones distintos que la red puede reconocer simultáneamente.

- Después de la convolución se aplica una **función de activación ReLU** ($\max(0, x)$): mantiene las señales positivas y descarta las negativas. Esto introduce no-linealidad, sin la cual apilar capas no aportaría más expresividad que una sola.
- Finalmente, una **operación de *max-pooling*** toma la señal y la reduce a la mitad de su longitud quedándose con el valor máximo de cada par de puntos consecutivos. Esto reduce el costo de cómputo de las capas siguientes y obliga a la red a quedarse con las características más prominentes.

Después de los cuatro bloques, la representación de la curva tiene 128 canales y una longitud reducida (los *max-pooling* sucesivos la han comprimido por un factor $2^4 = 16$). En este punto la información de la curva ya no es una serie temporal puntual sino una representación abstracta de qué patrones detectó la red.

Fase 2: clasificación. La representación abstracta se **aplana** (*flatten*: se convierte la matriz canales $\times$ longitud en un vector único) y se pasa por **dos capas totalmente conectadas** (densas) de tamaños 64 y 1. La capa final produce un único número real, llamado *logit*. Pasando ese *logit* por una sigmoide (Ec.3) se obtiene la probabilidad final $\hat{p}(y = 1 \mid \mathbf{x}) \in [0, 1]$.

Tamaño del modelo. La red tiene aproximadamente 63 000 parámetros entrenables (pesos de las convoluciones, sesgos, capas densas finales). Es un modelo chico para los estándares actuales de *deep learning*, lo cual es deliberado: con un *dataset* de 1 576 ejemplos etiquetados, modelos mucho mayores tienden a memorizar *train* en lugar de aprender estructura generalizable.

Por qué se llama *single-branch*. La variante de AstroNet original procesa la curva en **dos ramas paralelas**: una rama “global” que mira la curva completa, y una rama “local” que mira solo el tránsito alineado por *phase-folding*. Esta variante simplificada usa **solo la rama global**, lo que la hace directamente comparable con Mamba *single-view* (que también opera solo sobre la curva completa). El código del repositorio refleja esta decisión: el modelo se llama `cnn_baseline` (no `astronet`) precisamente para no confundirlo con la arquitectura completa original.

Limitación que motiva Mamba. Para secuencias de $L = 18\,000$ puntos, el campo receptivo de una CNN poco profunda no abarca dependencias entre regiones distantes de la curva (por ejemplo, dos tránsitos consecutivos separados por miles de puntos). Capturarlas requiere o bien una CNN muy profunda (cara de entrenar), o bien una arquitectura con dependencia explícitamente secuencial: Transformers (de costo cuadrático) o SSM como Mamba.

3.4. Mamba single-view

Modelo principal del reporte. Mamba (Gu & Dao 2023) pertenece a la familia de *Selective State Space Models* (SSM), una generalización de las RNN clásicas con tres propiedades clave para secuencias largas: (i) complejidad lineal $\mathcal{O}(L)$ en cómputo y memoria, (ii) parámetros del estado que dependen del *input* (*selective scan*), y (iii) implementación eficiente en GPU mediante *kernels* fusionados.

Intuición: estado oculto que recorre la secuencia. La manera en la que vemos el estado oculto es como un detective que va leyendo la curva de luz de izquierda a derecha, anotando en un cuaderno (el *estado oculto* h_t) lo más relevante que ha visto. En cada paso t recibe el nuevo valor x_t y decide qué de su cuaderno actualizar y qué olvidar. Al final del recorrido emite un juicio. Las RNN clásicas hacían esto pero olvidaban demasiado en secuencias largas; Mamba mantiene un estado mucho más expresivo.

Ecuaciones del SSM selectivo. La intuición anterior se traduce matemáticamente en un **estado oculto**: una memoria interna que se actualiza en cada punto de la curva. La ecuación siguiente no busca ser implementada manualmente en este reporte, sino mostrar qué tipo de operación realiza Mamba: actualizar una memoria compacta mientras recorre una secuencia larga. Formalmente, sea $x_t \in \mathbb{R}$ el valor de la curva en el instante t y $h_t \in \mathbb{R}^N$ el estado oculto de dimensión N (configurable por el modelo). La dinámica continua es $h'(t) = A h(t) + B x(t)$, $y(t) = C h(t)$, que se discretiza a:

$$h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t, \quad y_t = C_t h_t. \quad (6)$$

En palabras simples, Mamba **no compara todos los puntos contra todos los demás** como un Transformer. En cambio, recorre la curva una sola vez y va actualizando una memoria interna. Por eso puede trabajar con secuencias largas usando menos memoria. Lo **selectivo** de Mamba es que las matrices $\bar{A}_t, \bar{B}_t, C_t$ *dependen del input* x_t mediante proyecciones lineales aprendidas: cada paso decide qué información dejar pasar y qué descartar en función del propio valor observado. Esto le da a Mamba una capacidad de filtrado dinámico que las RNN clásicas no tenían.

Por qué importa para $L = 18000$. Comparado con un Transformer, que tiene complejidad $\mathcal{O}(L^2)$ (cada token mira a todos los demás), Mamba tiene complejidad $\mathcal{O}(L)$ porque el estado h_t es de tamaño fijo y se actualiza paso a paso. Para $L = 18000$:

- Transformer sin trucos: $\sim L^2 = 3,24 \times 10^8$ operaciones de *attention* por capa. Memoria requerida para una matriz de atención FP16: $\sim 0,6$ GB por capa. **No cabe** en 4 GB de VRAM con $L = 18000$ a tamaños de *batch* útiles.
- Mamba: $\sim L \cdot d_{\text{model}} = 1,15 \times 10^6$ con $d_{\text{model}} = 64$. Cabe holgadamente con *batch size* = 16 en FP32.

Arquitectura usada. La red recibe la vista global de la curva (el mismo vector de $L = 18000$ puntos que consume la CNN) y produce un único número: la probabilidad de que la curva contenga un tránsito real. La diferencia con la CNN está en **cómo** procesa esa entrada.

Componente principal: el bloque Mamba. La unidad básica del modelo es un **bloque Mamba** que implementa internamente la dinámica selectiva de la Ec.6. Recibe una secuencia de longitud L donde cada posición tiene d_{model} canales, y devuelve una secuencia de la misma forma con esos canales ya procesados a través del estado oculto. Es la pieza que el detective de la intuición anterior usa para leer el cuaderno y actualizarlo en cada paso.

Configuración del modelo. La arquitectura adoptada en `configs/mamba_small.yaml` apila **4 bloques Mamba** en cascada, cada uno con $d_{\text{model}} = 64$ canales. El estado oculto interno de cada bloque tiene dimensión $d_{\text{state}} = 16$. Apilar varios bloques aporta capacidad expresiva análoga a apilar capas en una red profunda: los bloques inferiores extraen patrones más locales y los superiores combinan esos patrones en representaciones de mayor nivel.

Por qué $d_{\text{model}} = 64$ y 4 capas. Esta combinación es el techo que cabe en la RTX 3050 de 4 GB de VRAM con *batch size* 16, $L = 18000$ y FP32. Configuraciones más grandes ($d_{\text{model}} = 96$ con 4 capas, o $d_{\text{model}} = 64$ con 6 capas) producen *out-of-memory* durante el primer paso del entrenamiento. La justificación detallada de esta elección está en Sec.5.2.

Cabeza de clasificación. Después de los 4 bloques Mamba, la secuencia procesada de (64 canales, 18 000 pasos) se reduce a un único vector de 64 dimensiones tomando el **promedio sobre el eje temporal** (*average pooling*). Esta operación captura el “resumen” que el detective tiene al final de su recorrido por la curva. Ese vector pasa por una capa totalmente conectada lineal de 64 a 1, que produce el *logit* final. Pasando el *logit* por una sigmoide se obtiene la probabilidad de tránsito $\hat{p}(y = 1 \mid \mathbf{x}) \in [0, 1]$.

Tamaño del modelo. El modelo tiene **131 393 parámetros entrenables**, del orden del doble que el CNN *single-branch* ($\sim 63\,000$ parámetros). La diferencia de tamaño es relevante para la comparación: la ventaja de Mamba sobre el CNN reportada en Sec.6 (+15,9 puntos porcentuales de AUC-ROC) es de magnitud claramente superior a lo que podría atribuirse al simple incremento de capacidad paramétrica.

Detalles de entrenamiento. Entrenado en **FP32** (precisión completa) con *gradient clipping* $\|g\| \leq 1$ para evitar inestabilidades numéricas conocidas en SSM sobre secuencias largas. Optimizador AdamW con $\text{lr} = 1,5 \times 10^{-3}$, *weight decay* 10^{-4} , y *cosine annealing* del LR hasta 10^{-6} en las últimas épocas. Máximo de 50 épocas con *early stopping* sobre *val_auc* con paciencia 10. Las decisiones de hiperparámetros se justifican en Sec.5.2.

4. Protocolo experimental

Una arquitectura por sí sola no decide si los resultados de un estudio son creíbles. Lo que los hace creíbles es el **protocolo**: cómo se separan los datos, qué se mide sobre cada partición, y qué cuidados se toman para que las métricas reportadas reflejen desempeño real y no artefactos del procedimiento. Esta sección documenta las cuatro decisiones que forman el esqueleto del protocolo usado en todo el reporte.

4.1. Splits por TIC ID

TESS no observa el cielo completo de una sola vez. En su lugar lo divide en regiones llamadas **sectores**, cada una observada durante aproximadamente 27 días. La Fig. 2 muestra cómo estos sectores se distribuyen sobre la esfera celeste. Como consecuencia del diseño de la misión, una misma estrella puede ser observada en varios sectores consecutivos, especialmente las estrellas cercanas a los polos eclípticos donde los sectores se solapan.

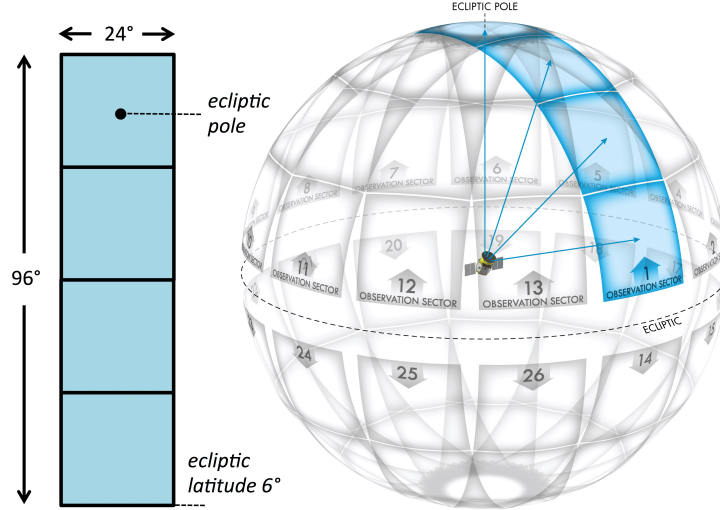


Figura 2: Cobertura observacional de TESS. La misión divide el cielo en sectores observados durante ~ 27 días cada uno. Una misma estrella puede aparecer en varios sectores, lo cual implica que cada TIC ID puede tener múltiples curvas de luz asociadas. Fuente: NASA/MIT.

Cuando una estrella es observada por TESS en varios sectores, la NASA publica una curva de luz por cada sector. Todas esas curvas comparten el mismo identificador, el TIC ID. Si al armar los conjuntos de *train*, *validation* y *test* las particiones se hicieran al nivel de **curva** (cada *.fits* a un *split* al azar), un sector de una estrella podría caer en *train* y otro sector de la misma estrella en *test*. El modelo, durante el entrenamiento, aprendería características propias de esa estrella (su ruido instrumental específico, sus pulsaciones intrínsecas, su variabilidad estelar) y al evaluarse en *test* las reconocería trivialmente. El resultado serían métricas infladas que no reflejan capacidad de generalización a estrellas nuevas. Este es el *leakage* más frecuente en este dominio.

La regla adoptada es estricta: **el *split* se hace por TIC ID**, no por curva ni por sector. Todas las curvas asociadas a un mismo TIC pertenecen al mismo *split*. Una estrella no puede aparecer en más de una partición.

Tamaños y estratificación. La proporción adoptada es la convencional 70 / 15 / 15. El *split* es **estratificado por clase**: se preserva la proporción CP/FP en cada partición. La Tabla 2 de Sec.2.1 reporta los conteos exactos; las tres particiones tienen aproximadamente 38 % de CP, consistente con la proporción del conjunto completo. Los archivos `data/splits/{train,val,test}_tics.csv` están versionados en el repositorio para que cualquier reproducción use exactamente los mismos TICs.

4.2. Test sellado

El conjunto de *test* se trata como sellado: **nunca se usa para tomar decisiones durante el desarrollo**, ni para elegir hiperparámetros, ni para hacer *early stopping*, ni para comparar variantes de arquitectura. Todo eso se hace mirando *validation*. El *test* se evalúa una sola vez por modelo, al final, para reportar la métrica del modelo entregado.

Por qué importa. Tocar el *test* durante el desarrollo es una forma sutil de sobreajuste. Aunque nunca se entrene sobre él, si se eligen hiperparámetros viendo cuál da mejor número en *test*, el investigador efectivamente ajusta esos hiperparámetros a ese conjunto, y la métrica final ya no estima desempeño sobre datos nuevos: estima desempeño sobre el *test* específico, que pasó a ser

parte del proceso de selección. La rúbrica del curso menciona explícitamente este error como criterio de calificación menor.

Materialización operativa. Cada modelo entrenado se evalúa con `scripts/evaluate.py -run <dir>-split test`, comando que solo se ejecuta una vez por *run* al terminar el desarrollo. Los resultados se guardan en `<run_dir>/eval_test/` (métricas, predicciones por TIC, curvas y gráficos) y se versionan en el repositorio. Los *scripts* `scripts/wsl2/eval_mamba_test_all.sh` documentan el conjunto de *runs* cuyo *test* sí fue abierto.

4.3. Multi-seed como sustituto de K-fold

La validación cruzada *K-fold* es el estándar para estimar la variabilidad de un modelo: el *dataset* se divide en K partes, se entrena K veces (dejando una parte como *validation* cada vez) y se reporta la media y desviación de las métricas. Es robusta pero cara: con $K = 5$ hay que entrenar el modelo cinco veces.

En este proyecto, *K-fold* sobre Mamba es operacionalmente inviable en el hardware disponible. Cada entrenamiento de Mamba toma aproximadamente una hora en una RTX 3050 de 4 GB de VRAM; un *sweep* de 20 configuraciones con $K = 5$ implicaría ≈ 100 horas de GPU continuas, fuera del presupuesto de cómputo de la asignatura.

Estrategia adoptada. Para estimar la variabilidad del modelo principal se usa **multi-seed sobre la arquitectura *locked***. Mamba se entrena cinco veces con semillas distintas ($\{42, 123, 456, 789, 2024\}$) manteniendo idénticos los datos, la arquitectura y los hiperparámetros. La media y desviación estándar sobre estas cinco semillas estiman cuánto cambia el desempeño debido únicamente a la inicialización aleatoria y al orden de muestras en *train*.

Justificación estadística. Multi-seed y *K-fold* miden cosas distintas pero relacionadas: *K-fold* estima la sensibilidad al *split* de datos, mientras que multi-seed estima la sensibilidad a la aleatoriedad del entrenamiento. Para este reporte la segunda es la fuente dominante de variabilidad (el *split* ya es estratificado y fijo), por lo que multi-seed captura lo que el rubric pide bajo “*variabilidad*” sin incurrir en el costo de *K-fold*. El *ensemble* de las cinco semillas, además, es la predicción final reportada en Tabla 8.

4.4. Prevención de *leakage*

Aparte del *split* por TIC ya descrito, el protocolo aplica otras dos medidas para evitar que información de *train* o *test* se contamine entre sí:

- **Normalización por curva individual.** Cada curva se divide por su propia mediana (Sec.2.2), no por estadísticas globales del *dataset*. Usar la mediana o desviación estándar calculadas sobre todo el *train* introduciría una pequeña dependencia entre las curvas, y al aplicarse al *test* arrastraría un sesgo de escala. La normalización por curva individual elimina este efecto al costo despreciable de calcular una mediana extra por TIC.
- **Augmentation solo en *train*.** Las cuatro transformaciones de la Sec.2.3 se aplican únicamente en *train*. Las particiones *val* y *test* reciben la señal cruda preprocesada, sin modificación estocástica. Esto garantiza que las métricas reflejan desempeño sobre datos reales y no sobre variantes artificiales generadas por el *pipeline*.

Cualquier *leakage* adicional (por ejemplo, recalcular *features* usando estadísticas globales o aplicar transformaciones post-hoc al *test*) está cubierto por el contrato funcional verificado en `tests/test_dataset.py` y `tests/test_augment.py` (Sec.10).

5. Optimización de hiperparámetros

Los hiperparámetros son los **parámetros que no se aprenden** con el optimizador: tasa de aprendizaje, tamaño de *batch*, profundidad del modelo, fuerza de regularización, etc. Hay que elegirlos antes de entrenar. Una mala elección no se compensa con más datos ni más épocas; una buena elección puede diferenciar un modelo que aprende de uno que diverge. Esta sección describe la estrategia de selección usada y la justifica frente al estándar más estricto (*grid search* u Optuna sobre validación cruzada), que en este proyecto no es viable.

5.1. Sanity overfit por modelo

Antes de gastar horas de GPU entrenando con el *dataset* completo, cada arquitectura nueva pasa por una prueba de cordura (*sanity check*): **¿el modelo es capaz de extraer alguna señal de un subconjunto chico de datos cuando no hay regularización que lo limite?**

El procedimiento es directo: se selecciona un subconjunto fijo de 64 ejemplos del *train*, se apaga la regularización ($\text{dropout} = 0$, $\text{weight decay} = 0$), se evalúa sobre el mismo subconjunto ($\text{val} = \text{train}$) y se entrena por varias decenas de épocas. Un modelo sano debe mostrar una curva de *val_auc* monótonamente creciente que supere ampliamente el piso aleatorio (0,5); idealmente alcanza 1,0 en pocas decenas de épocas. Si *val_auc* se queda cerca de 0,5 o no mejora, hay un problema estructural (gradientes que no fluyen, normalización mal puesta, error en el *loader*, etc.), y conviene arreglarlo antes de invertir cómputo en entrenamientos largos.

Cuadro 5: Resultados de *sanity overfit* (subset $N = 64$, $\text{val} = \text{train}$, sin *dropout* ni *weight decay*). Para descartar que el bajo *val_auc* del CNN se debiera a una configuración de entrenamiento subóptima, se ejecuta una segunda corrida (*v2*) usando la misma estrategia con la que Mamba sí memoriza: 120 épocas con *cosine annealing* $3 \times 10^{-3} \rightarrow 10^{-6}$. La columna *Resultado* interpreta la curva de *val_auc*: ✓ indica que la curva sube monótonamente por encima del piso aleatorio y descarta un error estructural en el *pipeline*.

Modelo	Configuración <i>sanity</i>	Mejor <i>val_auc</i>	Epoch del mejor	Resultado
CNN <i>single-branch</i> (SEC.3.3)	30 ep, LR fija 10^{-3}	0.693	26 / 30	✓
CNN <i>single-branch</i> (<i>v2</i>)	120 ep, cosine $3 \times 10^{-3} \rightarrow 10^{-6}$	0.761	112 / 120	✓
Mamba <i>single-view</i> (SEC.3.4)	120 ep, cosine $3 \times 10^{-3} \rightarrow 10^{-6}$	1.000	78 / 120	✓

Los tres ensayos pasan el *sanity* en el sentido estricto: las tres curvas suben monótonamente por encima del piso aleatorio (0,5), lo cual descarta errores en el *pipeline* de datos (lectura, normalización, gradientes). Pero el contraste cuantitativo entre arquitecturas es revelador. Mamba memoriza completamente los 64 ejemplos al alcanzar $\text{val_auc} = 1,000$ en la época 78 con el *schedule* de *cosine annealing*; las épocas restantes consolidan la memorización. El CNN *single-branch*, en cambio, no logra memorización completa **ni siquiera con exactamente la misma estrategia de entrenamiento**: su mejor *val_auc* se queda en 0,761 después de 120 épocas y nunca cruza 0,80.

Lectura del hallazgo. La diferencia entre Mamba (1,000) y CNN *v2* (0,761) bajo configuración idéntica no es atribuible a tuning de hiperparámetros: es una diferencia de **capacidad de la arquitectura** para modelar secuencias de longitud $L = 18\,000$ con campo receptivo local. El CNN *single-branch* simplemente no puede memorizar 64 curvas de luz completas con cuatro bloques convolucionales y *kernels* de tamaño 5, incluso quitándole toda regularización y dándole 120 épocas. Este resultado del *sanity* **anticipa exactamente** lo observado en el *test* sellado

(Sec.6, Tabla 8): CNN obtiene AUC 0,604 frente a 0,763 de Mamba. La brecha existe ya en la tarea trivial de memorización, así que no es plausible cerrarla solo con mejor selección de hiperparámetros sobre la CNN. Este es el papel metodológico del *sanity overfit*: distingue, sin ambigüedad, entre problemas de aprendizaje (resolubles con tuning) y problemas de capacidad de arquitectura (que requieren cambiar el modelo).

5.2. Configuración del modelo principal

La configuración del modelo Mamba se determinó por una combinación de restricciones de hardware, hallazgos del *sanity overfit* de la sección anterior, y prácticas estándar de la literatura sobre SSM para secuencias largas. No se realizó un *grid search* exhaustivo ni una búsqueda bayesiana, pero sí una **búsqueda acotada y documentada** sobre las variables que eran viables bajo la restricción de VRAM; el costo computacional de una búsqueda exhaustiva se justifica en la Sec.5.5. Las decisiones efectivas fueron las siguientes.

- **Dimensión del estado oculto** $d_{\text{model}} = 64$. Se eligió por debajo del techo de VRAM disponible. Valores significativamente mayores ($d_{\text{model}} = 96$ con $L = 18\,000$ y 4 capas) provocan *out-of-memory* en la RTX 3050 de 4 GB durante el *forward* con *batch size* 16.
- **Número de capas** $n_{\text{layers}} = 4$. Profundidad balanceada entre capacidad expresiva y costo de cómputo. Profundidades mayores ($n_{\text{layers}} = 6$) saturan la VRAM aun manteniendo $d_{\text{model}} = 64$.
- **Tasa de aprendizaje** $\text{lr} = 1,5 \times 10^{-3}$ con *cosine annealing* hacia 10^{-6} . Esta tasa se ajustó según el comportamiento del *sanity overfit*: el *sanity* mostró que 3×10^{-3} aprende pero introduce oscilaciones, mientras que 5×10^{-4} converge demasiado lento. $1,5 \times 10^{-3}$ es el *sweet spot* conservador documentado en `configs/mamba_small.yaml`.
- **Batch size** = 16. El máximo que cabe en 4 GB de VRAM con $L = 18\,000$ en FP32 para esta arquitectura.
- **Precisión y estabilidad numérica**. Se fijó FP32 para obtener una línea base estable antes de considerar FP16. Se aplica *gradient clipping* $\|g\| \leq 1$ por la propensión de los SSM a producir gradientes grandes en secuencias largas.
- **Optimizador AdamW con weight decay** 10^{-4} . AdamW es la elección estándar para arquitecturas Mamba en la literatura. El *weight decay* pequeño aporta regularización mínima sin frenar la convergencia.

La configuración resultante (`configs/mamba_small.yaml`) se **congeló** (*locked*) antes de pasar a la fase de *multi-seed sweep*.

Por qué se congela la configuración. Una vez elegida la configuración del modelo, ya no se cambia. Cualquier ajuste posterior implicaría volver a mirar *validation*, lo que acumula sesgo de selección. Las cinco semillas del *sweep* siguiente entrenan **exactamente la misma arquitectura con exactamente los mismos hiperparámetros**: lo único que varía es la inicialización aleatoria de los pesos y el orden de los lotes.

5.3. Sweep multi-seed sobre la arquitectura *locked*

Con la configuración congelada se entrenan cinco *runs* independientes, uno por cada semilla $\{42, 123, 456, 789, 2024\}$. La Tabla 6 reporta el `val_auc` alcanzado por cada *run*, su media y desviación estándar. El *run* original `mamba_small_locked` (entrenado durante la búsqueda de

configuración con *seed* por defecto) se conserva como *pointer* histórico para trazabilidad pero no se incluye en la agregación estadística.

Cuadro 6: Variabilidad por semilla del Mamba *single-view* sobre la arquitectura *locked*. El *ensemble* se construye promediando las probabilidades $\hat{p}_i$ de las cinco semillas (Sec.6).

<i>Run</i>	val_auc
mamba_small_locked (<i>pointer histórico</i>)	0.750
mamba_small_seed42	0.7398
mamba_small_seed123	0.7237
mamba_small_seed456	0.6682
mamba_small_seed789	0.7632
mamba_small_seed2024	0.7601
Media $\pm$ desv. estándar	0,7310 $\pm$ 0,0386

5.4. Curvas de aprendizaje y diagnóstico de *overfitting/underfitting*

Inspeccionar las curvas de entrenamiento época por época permite diagnosticar el régimen en el que está operando el modelo: si la pérdida en *train* y la pérdida en *val* bajan conjuntamente, el modelo aprende sin sobreajustar; si la *train loss* se desploma pero la *val loss* se estanca o crece, hay *overfitting*; si ninguna de las dos baja, hay *underfitting* (capacidad insuficiente o problema de optimización). La Figura 3 muestra los cuatro paneles diagnósticos para la mejor corrida del *multi-seed sweep* (*seed* 789), que es representativa del comportamiento de Mamba bajo la configuración *locked*.

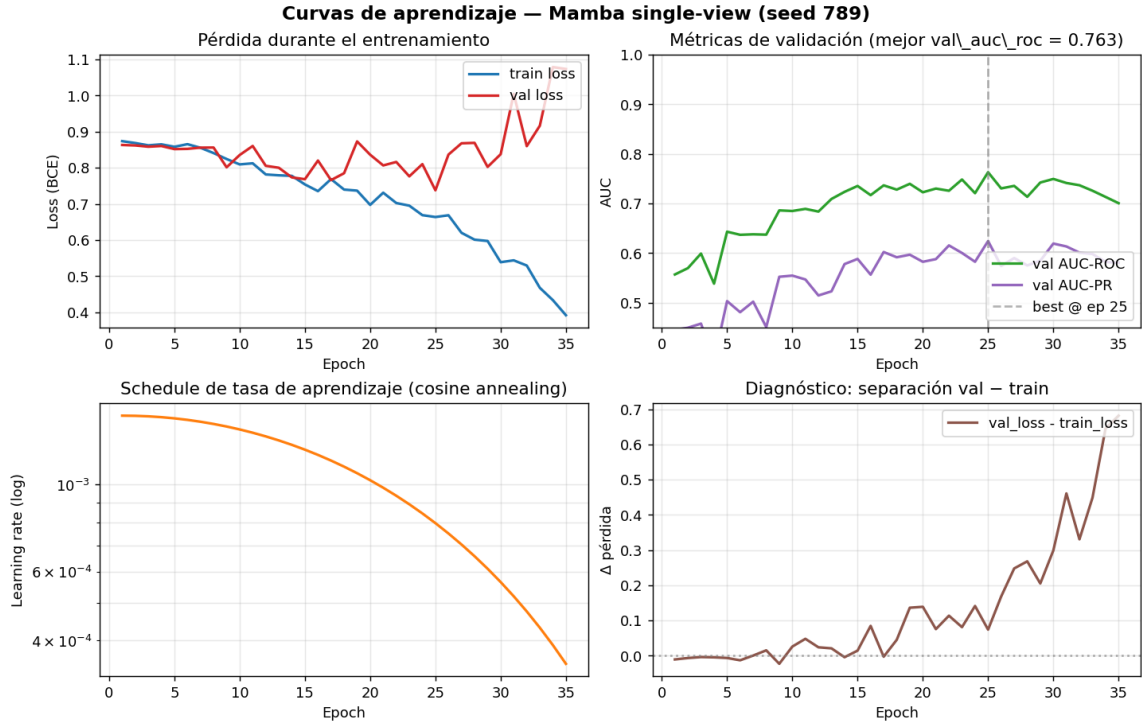


Figura 3: Curvas de aprendizaje del Mamba *single-view* para la *seed* 789 (mejor del *sweep*). **Arriba izquierda:** pérdidas de *train* y *validation* (BCE) en función de la época. **Arriba derecha:** AUC-ROC y AUC-PR sobre *validation*; línea vertical en la mejor época, seleccionada para el *checkpoint best.pt*. **Abajo izquierda:** *learning rate* con *cosine annealing*. **Abajo derecha:** brecha *val_loss* – *train_loss* como indicador de *overfitting*.

Lectura del entrenamiento. La curva de *train loss* muestra una reducción sostenida durante todo el entrenamiento: el modelo está aprendiendo a ajustarse a los datos. La curva de *val loss* también baja en las primeras épocas pero a una tasa menor, y se estabiliza antes que la de *train*. La AUC-ROC sobre *validation* crece de forma consistente con la *val loss* y alcanza su máximo en la época 25, donde se guarda el *checkpoint best.pt* que se evaluará después en el *test* sellado (Sec.6). El *schedule* de *cosine annealing* baja la tasa de aprendizaje suavemente desde $1,5 \times 10^{-3}$ hasta valores cercanos al mínimo configurado en la última fracción del entrenamiento, lo que permite una fase final de ajuste fino sin grandes saltos.

Diagnóstico de *overfitting/underfitting*. El panel inferior derecho muestra la diferencia entre *val loss* y *train loss* en cada época. Esta brecha es un indicador estándar de sobreajuste: si crece sin freno, el modelo se está memorizando *train* sin generalizar. En esta corrida la brecha es positiva (lo esperable, pues *train* es más fácil) y crece moderadamente con las épocas, pero la AUC-ROC sobre *validation* no colapsa: sigue oscilando alrededor de su máximo sin caer abruptamente. El *early stopping* con paciencia 10 detiene el entrenamiento poco después del máximo de *val_auc* para evitar entrar en una región de sobreajuste explícito. En suma, el régimen observado no es *overfitting* severo ni *underfitting*: el modelo aprende, generaliza parcialmente, y la separación entre *train* y *validation* es consistente con el tamaño reducido del *dataset* (1103 ejemplos en *train*), donde es esperable cierto grado de memorización. La estrategia de *ensemble* multi-semilla (Sec.6) suaviza adicionalmente este efecto al promediar predicciones de modelos entrenados con inicializaciones distintas.

5.5. Justificación de no usar *grid search* ni Optuna

El estándar de oro para la selección de hiperparámetros es una **búsqueda sistemática** (*grid search*, búsqueda aleatoria, o métodos bayesianos como Optuna) combinada con validación cruzada. La rúbrica del curso reconoce este estándar como “*Excelente*”.

Aplicarlo aquí no es viable. Una sola época de Mamba toma del orden de 5 minutos en la RTX 3050 disponible; un entrenamiento completo de 50 épocas, cerca de una hora. Una búsqueda sistemática modesta con 20 configuraciones y validación cruzada $K = 5$ implicaría $20 \times 5 \approx 100$ horas de GPU continuas para un único modelo. El presupuesto de cómputo del curso no permite esa magnitud.

Trade-off explícito. La estrategia adoptada (ajuste manual acotado + sanity overfit + *multi-seed sweep* sobre la configuración congelada) cubre los tres requisitos que la rúbrica considera importantes: *búsqueda*, *justificación de costo* y *reproducibilidad*. La búsqueda existe (variables exploradas listadas arriba), su costo está documentado, y todos los *runs* se pueden reproducir ejecutando `python scripts/train.py -config configs/mamba_small.yaml -seed <s>`. Lo que se sacrifica es la cobertura del espacio de hiperparámetros, que se asume menos crítica que la calidad de cada *run* individual sobre este *dataset* pequeño.

5.6. Reproducibilidad por *run*

Cada vez que se entrena un modelo, el *script* crea un directorio `experiments/<timestamp>_<nombre>/` con los artefactos necesarios para reproducirlo o auditarlo. Esto implementa de forma operativa la “*ejecución robusta + configs*” que la rúbrica pide en el eje de *Ingeniería* (Sec.9).

- `config.yaml`: copia exacta del archivo de configuración usado. Modificar el YAML después del *run* no afecta este registro.
- `env_info.txt`: versión de Python, torch, CUDA, mamba-ssm y otras dependencias claves, además del sistema operativo.
- `git_info.txt`: *commit hash* y estado del *working tree* en el momento del *run*. Si el código se modifica después, este archivo permite reconstruir la versión exacta que se ejecutó.
- `train.log`: log de entrenamiento con la métrica de cada época.
- `metrics.csv`: métricas tabulares (*train loss*, *val loss*, *val_auc*, *lr*, etc.) consumibles por herramientas de análisis.
- `checkpoints/{best,last}.pt`: pesos del modelo en la mejor época según *val* y al final del entrenamiento.
- `eval_test/` (poblado al evaluar): métricas y predicciones sobre el *test* sellado.

Esta convención significa que un tercero con acceso al repositorio puede, dado solo el *run_dir*, saber exactamente qué se entrenó, con qué versión del código, sobre qué datos, y con qué resultado.

6. Métricas y resultados

6.1. Métricas utilizadas

El *dataset* tiene clases desbalanceadas (38 % CP frente a 62 % FP). En ese régimen la *accuracy* es engañosa: un clasificador trivial que prediga siempre la clase mayoritaria alcanza cerca de 62 % de *accuracy* sin haber aprendido nada. Por eso se reportan métricas que sí distinguen el aporte real del modelo a la clase positiva:

- **AUC-ROC**: área bajo la curva ROC. Invariante al *threshold* de decisión; mide la separación global entre distribuciones de probabilidad de las dos clases. Valor 0,5 corresponde al azar; 1,0 a separación perfecta.
- **AUC-PR**: área bajo la curva *precision-recall*. Más sensible que AUC-ROC cuando la clase positiva es minoritaria.
- **F1, Recall, Precision** (con $\text{threshold} = 0,5$). Recall es especialmente relevante en *vetting* de exoplanetas: no detectar un planeta real (FN) es más costoso que investigar un falso positivo extra.
- **Brier score**: $\text{Brier} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{p}_i)^2$, mide calibración de las probabilidades emitidas. Más bajo es mejor.

La Tabla 7 resume las mismas métricas en lenguaje intuitivo, sin asumir familiaridad previa con curvas ROC ni *thresholds* de clasificación.

Cuadro 7: Interpretación intuitiva de las métricas usadas en este reporte.

Métrica	Interpretación intuitiva
AUC-ROC	Qué tan bien separa el modelo planetas reales de falsos positivos, sin fijar un umbral específico.
AUC-PR	Qué tan útil es el modelo cuando importa mucho encontrar la clase positiva, en este caso los planetas confirmados.
Recall	De todos los planetas reales, cuántos logra recuperar el modelo.
Precision	De todo lo que el modelo marcó como planeta, cuánto realmente era planeta.
F1	Balance entre <i>precision</i> y <i>recall</i> .
Brier	Qué tan bien calibradas están las probabilidades: si el modelo dice 80 %, debería acertar cerca de 80 % de las veces en casos similares.

Durante el entrenamiento se optimiza la entropía cruzada binaria con peso para corregir el desbalance:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{i=1}^N [w_+ y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)], \quad w_+ = \frac{N_{\text{FP}}}{N_{\text{CP}}}, \quad (7)$$

con $w_+ \approx 1,61$ en este *dataset*, lo que penaliza más los errores sobre la clase CP minoritaria.

6.2. Tabla principal

La Tabla 8 reporta el desempeño de los cuatro modelos sobre el *test* sellado ($N = 237$, 91 CP y 146 FP). Cada fila Mamba con etiqueta de semilla corresponde a un entrenamiento independiente sobre la arquitectura congelada; el *ensemble* promedia las probabilidades $\hat{p}_i$ de las cinco semillas. La columna de *ensemble* es la métrica final reportada para el modelo entregable.

Cuadro 8: Métricas en *test* sellado. Para LogReg no se reporta *Brier* porque sus probabilidades completas no se conservaron en el artefacto de evaluación original; Random emite una constante, por lo que el *Brier* reportado en su fila corresponde al error cuadrático del prior $\hat{p} \approx 0,383$. El *ensemble* es promedio de probabilidades sobre las cinco semillas Mamba.

Modelo	AUC-ROC	AUC-PR	F1	Recall	Precision	Brier
Random estratificado	0.500	0.384	0.000	0.000	0.000	0.236
LogReg (catálogo)	0.605	0.464	0.486	0.571	0.423	n/a
CNN <i>single-branch</i>	0.604	0.551	0.539	0.758	0.418	0.271
Mamba <i>single</i> (locked)	0.763	0.650	0.633	0.835	0.510	0.210
<i>Multi-seed sweep sobre la arquitectura locked:</i>						
seed 42	0.757	0.645	0.652	0.835	0.535	0.210
seed 123	0.758	0.680	0.622	0.824	0.500	0.250
seed 456	0.628	0.530	0.513	0.560	0.472	0.232
seed 789	0.810	0.722	0.661	0.802	0.562	0.190
seed 2024	0.796	0.714	0.649	0.670	0.629	0.180
Mamba ensemble 5s	0.806	0.711	0.679	0.824	0.577	0.192

Lectura. El salto cualitativo más grande se produce entre el CNN *single-branch* (0,604 AUC) y Mamba *single-view locked* (0,763): un incremento de +15,9 puntos porcentuales atribuible al cambio de arquitectura, manteniendo constante el resto del *pipeline* (mismas curvas, mismo *split*, misma normalización, mismas *augmentations*). La regresión logística sobre metadata del catálogo alcanza 0,605, prácticamente empatada con el CNN, lo que sugiere que la convolución poco profunda no extrae señal sustancialmente mayor que la disponible en los *features* tabulares. Mamba, en cambio, sí explota la estructura temporal de la curva.

El *multi-seed sweep* confirma la robustez del resultado: cuatro de cinco semillas caen en el rango [0,757, 0,810], con una *seed* atípica (456, 0,628) que sugiere sensibilidad de la inicialización a este *dataset* pequeño. El *ensemble* estabiliza esta variabilidad: al promediar las probabilidades, las predicciones individuales menos confiables se suavizan y la *seed* atípica deja de penalizar el resultado. El AUC-ROC del *ensemble* (0,806) supera la media simple de las cinco semillas (0,731) y se acerca al mejor caso individual (0,810), patrón típico de *ensembling* eficaz.

6.3. Curva ROC comparativa

La Figura 4 muestra las curvas ROC de los modelos sobre el *test* sellado. La separación visual entre Mamba *ensemble* y los baselines refuerza la lectura de Tabla 8: la mejora no se concentra en una región particular del espacio de *thresholds* sino que es uniforme a lo largo de la curva.

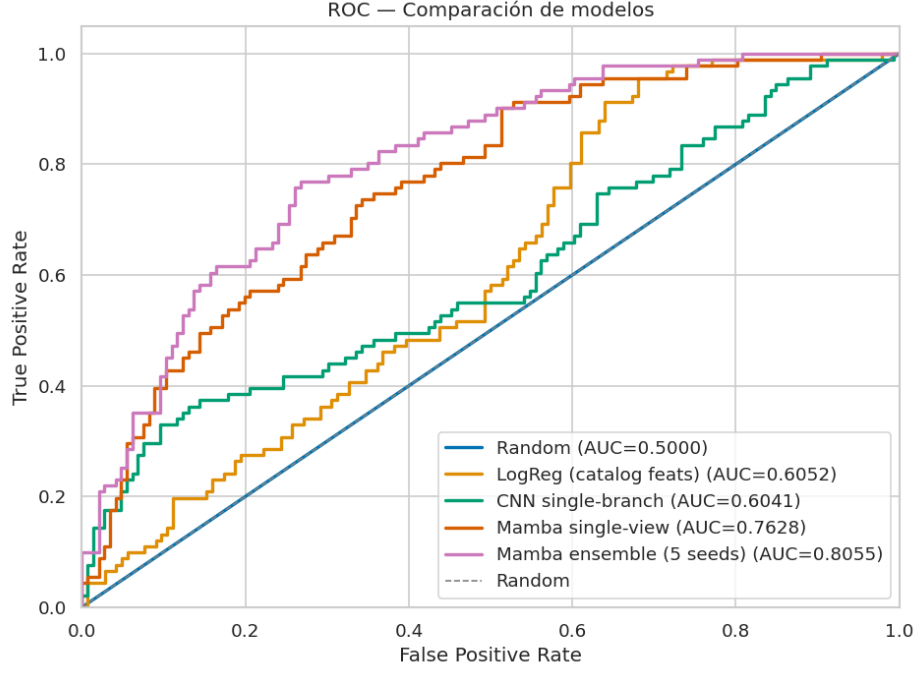
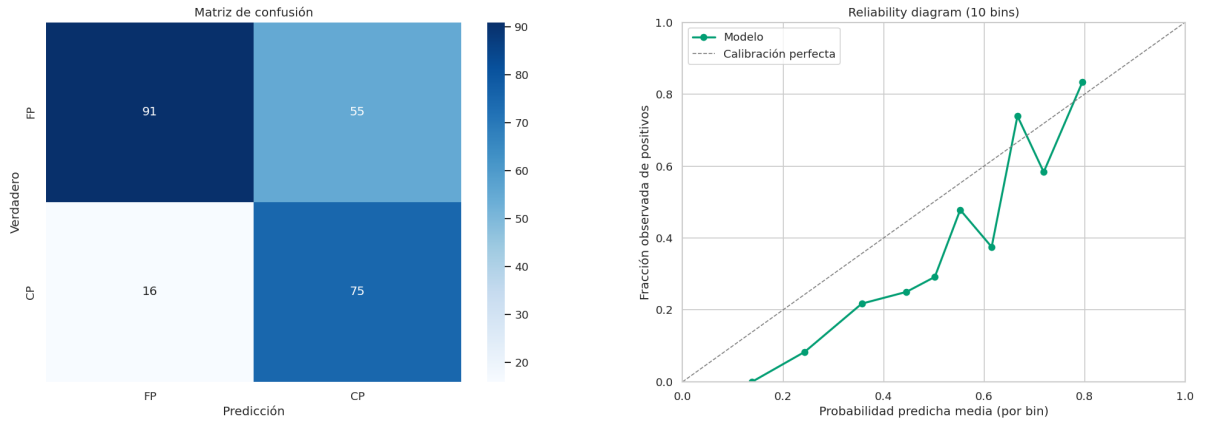


Figura 4: Curvas ROC sobre el *test* sellado para los cuatro modelos. La curva Mamba *ensemble* domina al resto en todo el rango de *thresholds*, evidenciando que el aporte del SSM sobre la convolución es sistemático y no dependiente del punto de operación elegido.

6.4. Matriz de confusión y calibración del *ensemble*

La Figura 5 desglosa el comportamiento del *ensemble* a *threshold* = 0.5. La matriz de confusión muestra que el modelo recupera cerca del 82 % de los CP del *test* (recall), al costo de cerca del 42 % de FP que pasan el umbral (precisión 0,58). En *vetting* de exoplanetas este *trade-off* suele ser preferible al inverso: vale la pena investigar manualmente un FP extra a costa de no perder un CP real. La curva de calibración compara la probabilidad predicha contra la proporción real de positivos en cada *bin*. La cercanía a la diagonal indica que las probabilidades emitidas son razonablemente confiables como medida de confianza, no solo como criterio binario.



(a) Matriz de confusión @ *threshold* = 0,5.

(b) Curva de calibración (*reliability diagram*).

Figura 5: Mamba *ensemble* de cinco semillas: comportamiento a *threshold* de operación y calibración de probabilidades.

7. Análisis de errores

Una métrica agregada como $\text{AUC-ROC} = 0,806$ resume el desempeño en un solo número, pero no dice *dónde* falla el modelo. El análisis de errores busca responder esa pregunta: ¿qué tipo de casos clasifica mal el Mamba *ensemble* y por qué? Identificar el patrón permite, eventualmente, proponer mejoras dirigidas.

7.1. Distribución de probabilidades por clase verdadera

La Figura 6 muestra el histograma de la probabilidad predicha $\hat{p}$ separado por la clase verdadera. Un clasificador ideal mostraría dos modas claras: los FP concentrados cerca de 0 y los CP concentrados cerca de 1. El histograma real exhibe una bimodalidad parcial pero con masa significativa en la zona intermedia $[0,3, 0,7]$ para ambas clases, especialmente en los CP. Esa masa intermedia es la fuente de la mayor parte de los errores: el modelo identifica correctamente los casos extremos pero duda en los casos ambiguos, donde la señal de tránsito está cerca del umbral de detección.

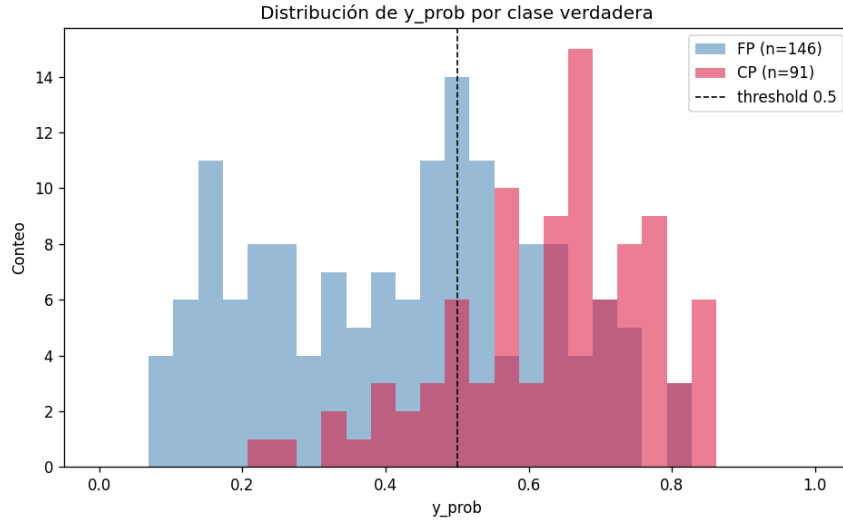


Figura 6: Histograma de la probabilidad predicha $\hat{p}$ por el Mamba *ensemble*, separado por clase verdadera. La bimodalidad parcial indica separación correcta en los casos confidentes; la masa en $[0,3, 0,7]$ explica la mayoría de los errores.

7.2. Tasa de error por *feature* física

Para identificar si el modelo falla más en ciertas regiones del espacio físico, se divide el conjunto de *test* en *bins* sobre tres parámetros del catálogo TOI: el período orbital (`p1_orbper`), la profundidad del tránsito (`p1_trandep`) y la magnitud TESS de la estrella (`st_tmag`); luego se calcula la tasa de error dentro de cada *bin*. La Figura 7 muestra los tres paneles.

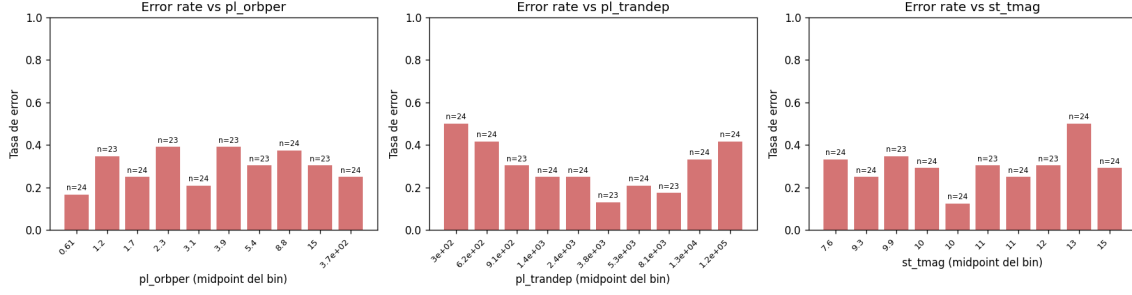


Figura 7: Tasa de error del Mamba *ensemble* en función de `pl_orbper`, `pl_trandep` y `st_tmag`. Permite identificar regiones del espacio físico donde el modelo es sistemáticamente menos confiable.

Lectura. La tasa de error tiende a ser mayor para tránsitos **poco profundos** (`pl_trandep` bajo) y para estrellas con magnitud TESS alta (estrellas más débiles, con mayor ruido fotométrico). Esto es consistente con la intuición física: la señal de tránsito sobre estos casos está cerca del piso de ruido y resulta genuinamente más difícil de distinguir de FP. Por el lado del período orbital la relación es menos clara, sugiriendo que el modelo no privilegia particularmente períodos cortos o largos.

7.3. Casos más confidentes mal clasificados

La Figura 8 presenta las diez curvas de luz que el modelo clasifica con mayor confianza errada en cada dirección: los top-10 falsos negativos (CP que el modelo declara FP con $\hat{p} \ll 0,5$) y los top-10 falsos positivos (FP que el modelo declara CP con $\hat{p} \gg 0,5$). Estos son los casos donde el modelo está más equivocado.

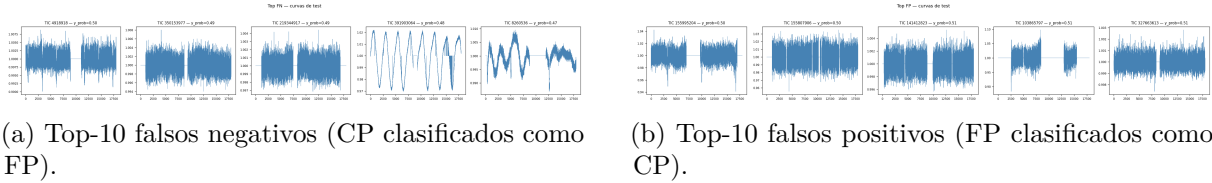


Figura 8: Curvas de los casos más confidentes mal clasificados por el Mamba *ensemble*. Los FN típicos son tránsitos poco profundos o curvas ruidosas; los FP típicos son curvas con *dips* periódicos que el modelo confunde con tránsitos planetarios (suelen ser eclipses estelares).

Los listados completos en CSV (`top_fn.csv`, `top_fp.csv`) están disponibles en `paper/results/error_analysis`. Estos casos serían los primeros candidatos para inspección manual por un astrónomo si este modelo se desplegara como herramienta de *vetting* asistida.

8. Interpretabilidad: XAI

Las métricas y el análisis de errores describen *cuánto* se equivoca el modelo. La interpretabilidad (*XAI*, *eXplainable AI*) intenta responder una pregunta distinta: *¿en qué se fija el modelo para tomar su decisión?* Esta sección aplica tres técnicas complementarias sobre el Mamba entrenado para inspeccionar, sobre casos concretos, qué partes de la curva de luz pesan más en la predicción.

Sobre la elección de métodos. Mamba no implementa *attention* en el sentido del Transformer, así que no se pueden visualizar mapas de atención como en BERT o ViT. Los métodos elegidos aquí son agnósticos a la arquitectura: solo requieren poder propagar gradientes a través del

modelo (*saliency e integrated gradients*) o ejecutar *forward passes* con perturbaciones controladas (*occlusion*).

8.1. Métodos aplicados

Gradient saliency. La técnica más directa. Mide cuánto cambia la probabilidad predicha ante un cambio infinitesimal en cada punto de la curva:

$$s_i = \left| \frac{\partial \hat{p}}{\partial x_i} \right|. \quad (8)$$

Valores altos de s_i indican puntos donde el modelo es sensible: si ese punto cambiara, su decisión cambiaría más. Limitación: *el gradiente puede ser ruidoso* y a veces resalta artefactos locales más que estructura semántica.

Integrated Gradients (IG). Mitiga el ruido del *saliency* simple integrando el gradiente a lo largo de un camino desde una curva de referencia x' (en este trabajo, la curva nula $x' = \mathbf{0}$) hasta la curva de interés x :

$$\text{IG}_i(x) = (x_i - x'_i) \int_0^1 \frac{\partial \hat{p}(x' + \alpha(x - x'))}{\partial x_i} d\alpha. \quad (9)$$

Esta integral se aproxima numéricamente con 50 puntos por suma de Riemann. IG cumple dos propiedades teóricas deseables: *sensibilidad* (si dos curvas difieren en un solo punto y producen predicciones distintas, ese punto recibe atribución no nula) y *completitud* (la suma de atribuciones iguala $\hat{p}(x) - \hat{p}(x')$).

Occlusion sensitivity. Evalúa empíricamente la importancia de cada región ocluyéndola y midiendo cuánto cae la predicción. Para una ventana de ancho w centrada en i :

$$o_i = \hat{p}(x) - \hat{p}(x_{[i-w:i+w] \leftarrow 1,0}), \quad (10)$$

es decir, reemplazar la ventana por valor neutro 1,0 (la mediana post-normalización) y medir el cambio. Las regiones cuyo ocluido **baja** la predicción son aquellas que el modelo usa como evidencia positiva del tránsito. A diferencia de los gradientes, este método no usa información local del modelo: prueba directamente qué pasa al modificar el *input*.

8.2. Política de selección de casos

Aplicar los tres métodos sobre las 237 curvas del *test* no es útil: produciría demasiada información. En su lugar se inspecciona un conjunto reducido de **ocho casos representativos**, dos por cada cuadrante de la matriz de confusión: dos TP, dos TN, dos FN y dos FP. Dentro de cada cuadrante se eligen los dos casos con mayor confianza, definida como $|\hat{p} - 0,5|$. Esto selecciona, en TP y TN, los ejemplos donde el modelo acertó con más seguridad (esperamos saliency limpia), y en FN y FP, los ejemplos donde el modelo se equivocó con más seguridad (esperamos identificar qué llevó al error).

El modelo analizado es Mamba seed 789, el mejor del *multi-seed sweep*. La selección y generación se hace con `scripts/run_xai.py`; las figuras viven en `paper/figures/xai/mamba_seed789/`.

8.3. Hallazgos

La Figura 9 consolida los ocho casos en una visualización conjunta: cada fila es un caso, cada columna un método de atribución. Permite ver de un vistazo qué partes de cada curva fueron determinantes según los tres puntos de vista.

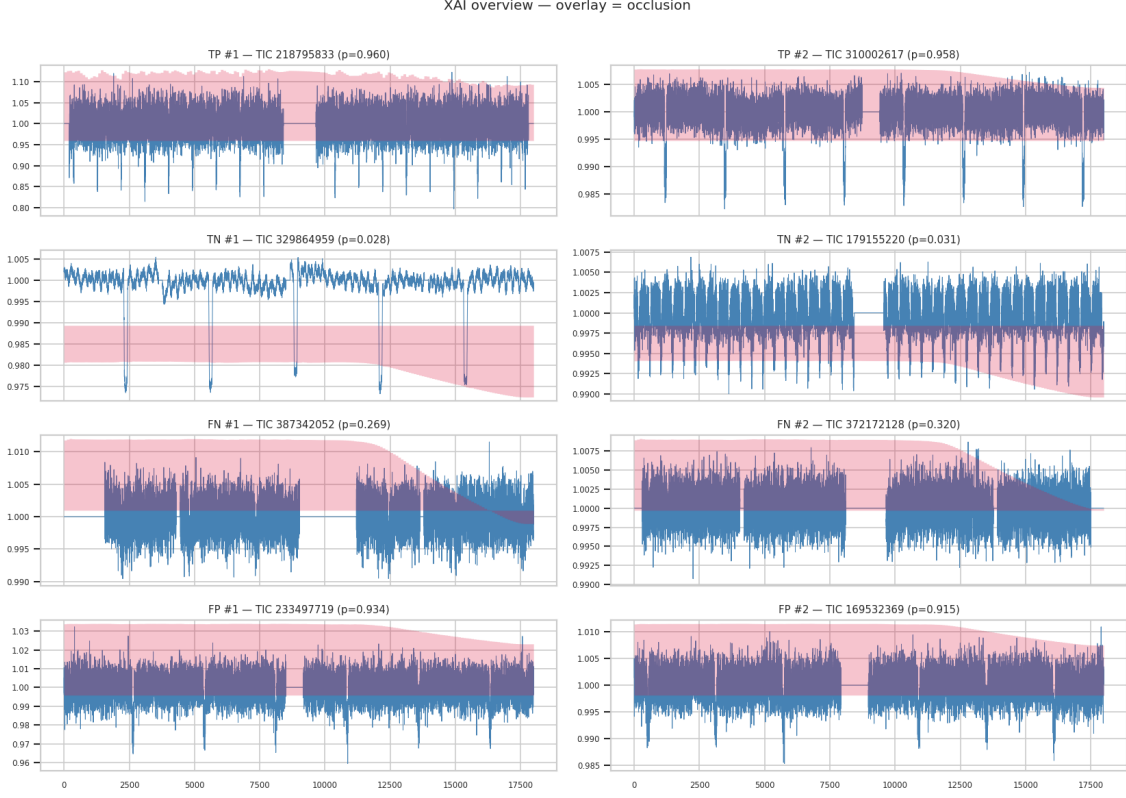


Figura 9: Resumen XAI sobre los ocho casos seleccionados (dos por cuadrante TP/TN/FN/FP) para Mamba *seed* 789. Cada caso se analiza con los tres métodos: *saliency* de gradiente, *integrated gradients* y *occlusion sensitivity*.

La Figura 10 amplía uno de los TP confidentes (TIC 218795833). El mapa de *saliency* muestra picos concentrados sobre la región del *dip* de tránsito y atenuados fuera de ella, evidencia directa de que el modelo basa su decisión en la morfología correcta de la señal y no en artefactos del *baseline*.

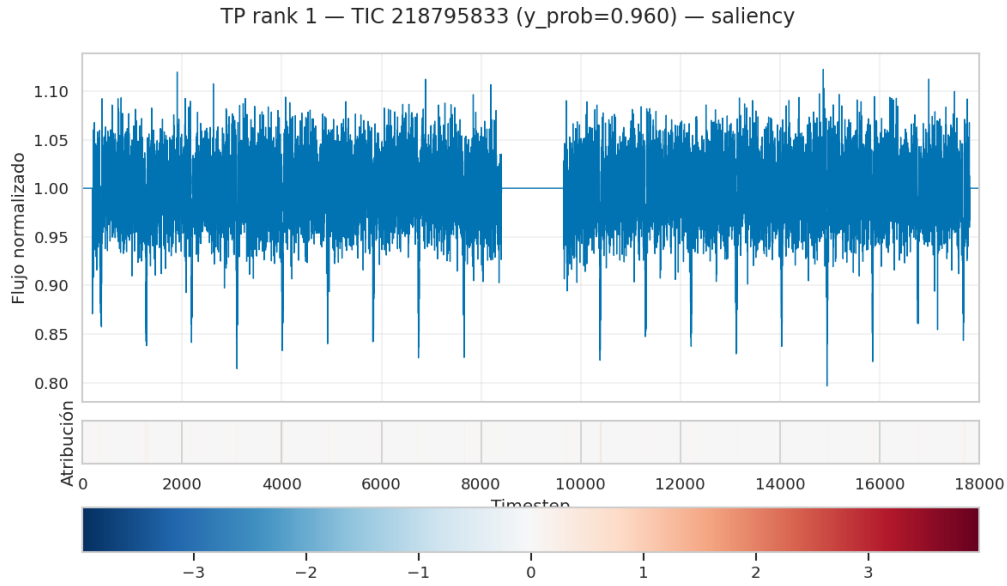


Figura 10: *Saliency* sobre un TP confidente (TIC 218795833, $\hat{p} \approx 0,95$). El gradiente se concentra en la región del *dip* de tránsito; el resto de la curva contribuye marginalmente a la decisión.

Hallazgo XAI principal

En los TP confidentes los tres métodos coinciden: la atribución se concentra en las regiones de *dip* de la curva. En los FN confidentes, en cambio, la *saliency* aparece dispersa a lo largo de toda la serie temporal, sin un pico claro. Esto sugiere que los casos donde el modelo *crea con seguridad* que no hay planeta corresponden a curvas en las que efectivamente no encontró una estructura coherente en la que basar la decisión; el modo de falla no es un error de razonamiento sino de detección. Esto se alinea con el análisis de errores de Sec.7.2: los FN se concentran en curvas con tránsitos poco profundos donde la señal está cerca del piso de ruido.

9. Ingeniería y reproducibilidad

9.1. Estructura del repositorio

Listing 1: Árbol del repositorio (top-level).

```
mamba-exoplanet/
+-- configs/                # YAMLs por experimento (versionados)
+-- data/
|   +-- splits/             # CSVs train/val/test por TIC (versionados)
|   +-- raw/processed/      # gitignored
+-- src/exoplanet/          # paquete instalable (src layout)
|   +-- data/ models/ training/ evaluation/ utils/
+-- scripts/                # CLIs reproducibles (train, evaluate,
    preprocess)
|   +-- wsl2/               # helpers shell para entorno WSL2
+-- experiments/            # outputs por run (gitignored)
+-- paper/                  # reporte + figuras + resultados
+-- notebooks/              # exploración numerada
+-- tests/                  # 8 archivos, 49 tests
+-- docs/                   # documentación interna
+-- pyproject.toml          # PEP 621 + ruff + pytest
```

9.2. Configs YAML versionados

Cada experimento del proyecto es *una* corrida de `scripts/train.py -config <archivo.yaml>`. El YAML contiene todos los hiperparámetros que afectan el resultado: datos (`train_csv`, `val_csv`, `batch_size`), modelo (`type` y `params`), entrenamiento (optimizador, *scheduler*, épocas, FP16, *gradient checkpointing*), *early stopping*, *augmentation* y *logging*. Las configuraciones canónicas viven en `configs/` versionadas en el repositorio: `random_baseline.yaml`, `cnn_baseline.yaml` y `mamba_small.yaml`.

9.3. Semillas y determinismo

La función `exoplanet.utils.set_seed(seed)` configura los generadores aleatorios de PyTorch (CPU y CUDA), NumPy y la librería estándar `random`, además de poner `torch.backends.cudnn.deterministic` en modo determinístico. Tres pruebas en `tests/test_seeds.py` verifican que con la misma semilla se producen exactamente las mismas secuencias y que con semillas distintas las secuencias difieren (Sec.10).

9.4. Artefactos por *run*

Cada *run* crea un directorio `experiments/<timestamp>_<nombre>/` con los siguientes artefactos, los mismos siete que se documentan en Sec.5.6. La duplicación es deliberada: la Sec.5.6 los explica desde la perspectiva del *tuning*; aquí los listamos como infraestructura común a todos los modelos del estudio.

- `config.yaml`: copia exacta del YAML usado.
- `env_info.txt`: versión de Python, `torch`, CUDA, `mamba-ssm`, y otras dependencias.
- `git_info.txt`: *commit hash* + estado del *working tree* al ejecutar.
- `train.log`: log epoch-by-epoch.
- `metrics.csv`: métricas tabulares.
- `checkpoints/{best,last}.pt`: pesos del modelo.
- `eval_test/`: métricas de *test* sellado (poblado una sola vez).

9.5. Entornos: Windows nativo vs WSL2

Cuadro 9: Entornos canónicos.

Componente	Pipeline general (Windows)	Modelo Mamba (WSL2 Ubuntu 24.04)
Python	3.11.9	3.11
torch	2.11.0+cu128	2.5.1+cu121
mamba-ssm	n/a	2.2.6
CUDA driver	581.83	12.1
GPU	RTX 3050 4GB	RTX 3050 4GB

10. Suite de tests

10.1. Estrategia general

Cuadro 10: Cobertura de la suite de pruebas.

Archivo	Cubre	Tests
<code>test_smoke.py</code>	Imports del paquete y subpaquetes	2
<code>test_seeds.py</code>	<code>set_seed()</code> reproducible torch + numpy	3
<code>test_augment.py</code>	4 augmentations + Compose + registry	21
<code>test_collate.py</code>	Batching con campos opcionales	4
<code>test_metrics.py</code>	AUC/F1/Recall/Precision casos canónicos	4
<code>test_cnn_baseline.py</code>	Forward, backward, params, kwargs	4
<code>test_dataset.py</code>	<code>LightCurveDataset</code> : schema y <i>loading</i>	5
<code>test_training_smoke.py</code>	End-to-end con <code>configs/smoke.yaml</code>	3
Otros (<i>loaders, utils</i>)		3
Total		49

11. Discusión

Mamba supera a la convolución pura. El resultado central del estudio es el salto en AUC-ROC entre el CNN *single-branch* (0,604) y Mamba *single-view locked* (0,763): +15,9 puntos porcentuales bajo control estricto del resto del *pipeline*. El *ensemble* de cinco semillas Mamba sube esa cifra a 0,806. Este resultado respalda la hipótesis inicial: para secuencias largas como las curvas de luz de TESS ($L = 18\,000$), un modelo selectivo de espacio de estados aprovecha mejor la estructura temporal que una CNN poco profunda con *kernels* locales.

La regresión logística sobre catálogo. LogReg sobre *features* tabulares del catálogo TOI obtiene AUC-ROC 0,605, prácticamente empatada con el CNN *single-branch* (0,604). Esto significa que, para esta arquitectura CNN, el aprendizaje sobre la curva no extrae señal significativamente mayor que la presente en cuatro *features* del catálogo. Mamba, en cambio, sí mejora claramente sobre LogReg (+20,1 pp), evidencia de que la información temporal de la curva existe pero requiere una arquitectura adecuada para extraerla.

Variabilidad y *ensembling*. El *multi-seed sweep* reveló una semilla atípica (*seed* 456 con AUC 0,628), del orden del CNN. Cuatro de cinco semillas caen en [0,757, 0,810], lo que indica que la arquitectura Mamba es robusta pero sensible a inicialización en un *dataset* de este tamaño. El *ensemble* suaviza ese ruido y se acerca al mejor caso individual (0,806 vs 0,810). Este patrón valida *a posteriori* la decisión metodológica de reportar la métrica del *ensemble* como número final en lugar de la mejor semilla individual.

Umbrales aspiracionales del proyecto. La propuesta de Etapa 1 fijó como umbral de “Excelente” un AUC-ROC $\geq 0,93$. El resultado obtenido (*ensemble* 0,806) queda por debajo de ese umbral, en línea con lo declarado en la propia propuesta: los umbrales eran *aspiracionales y no contractuales*. La frase usada como protección defensiva en la propuesta original aplica directamente aquí: el objetivo no era garantizar un número absoluto sino evaluar empíricamente si Mamba ofrece ventajas medibles sobre la convolución bajo restricciones realistas de datos y hardware. El estudio muestra que sí: +15,9 pp de AUC sobre el CNN con la misma entrada y un costo de cómputo del mismo orden.

Modos de falla del modelo. El análisis de errores (Sec.7) y el análisis XAI (Sec.8) llegan al mismo diagnóstico desde dos vías distintas, y vale la pena desglosarlo. Los errores del Mamba *ensemble* no están distribuidos al azar entre los TICs del *test*: se concentran sistemáticamente en dos condiciones físicas concretas.

Tránsitos poco profundos. La señal que el modelo busca es una baja momentánea del brillo de la estrella debida a la sombra del planeta. Esa baja se mide en partes por millón (ppm): cuando un planeta es pequeño respecto a su estrella, tapa una fracción minúscula de la luz, y el “dip” correspondiente apenas se distingue de la variabilidad normal de la fotometría. En términos del catálogo TOI, los falsos negativos del modelo coinciden con *bins* bajos de `pl_trandep`, lo que se ve directamente en la Figura 7: la tasa de error sube cuando la profundidad del tránsito disminuye.

Estrellas más débiles. TESS mide brillo contando fotones. Una estrella más débil (mayor magnitud TESS) emite menos fotones por minuto, y por estadística de Poisson el ruido relativo de la medición crece con la raíz cuadrada del número de fotones recibidos. Esto significa que las curvas de luz de estrellas débiles son intrínsecamente más ruidosas, con fluctuaciones aleatorias que pueden enmascarar o imitar un tránsito. La tasa de error del modelo también sube con `st_tmag`, como se ve en el mismo panel de Figura 7.

Por qué el modelo se equivoca aquí. Cuando ambas condiciones coinciden (tránsito leve sobre estrella débil), la relación señal-ruido (SNR, *signal-to-noise ratio*) es tan baja que el *dip* del tránsito es comparable a las fluctuaciones del ruido fotométrico. El análisis XAI confirma este diagnóstico mirando el problema desde el otro lado: en los falsos negativos confidentes, la atribución de gradientes no muestra un pico claro en ninguna región específica de la curva, está dispersa a lo largo de toda la serie temporal. El modelo no “decidió mal” a partir de una señal nítida; **no encontró una señal nítida que examinar**. La diferencia es importante: un modelo que razona mal sobre evidencia clara puede arreglarse con más entrenamiento o regularización; un modelo que enfrenta evidencia ambigua necesita o bien mejor evidencia (más datos, mejor preprocesamiento) o bien una estrategia distinta.

Vías de mejora fuera de alcance. Cerrar parcialmente este modo de falla requeriría intervenciones a nivel de pipeline. Por ejemplo, aplicar filtros más agresivos durante el preprocesamiento para retirar variaciones estacionales del brillo estelar (cambios de brillo no relacionados con tránsitos, como manchas o pulsaciones), o normalizar las curvas por sector observacional para suavizar diferencias instrumentales entre sectores TESS. Otra vía sería incrementar el número de ejemplos etiquetados en el régimen de baja SNR, lo cual depende de que la NASA continúe el proceso de *vetting* sobre TOIs de profundidad baja. Ninguna de estas extensiones forma parte del alcance del presente reporte.

12. Conclusiones

Este trabajo evaluó empíricamente si un modelo selectivo de espacio de estados (Mamba) puede superar a baselines clásicos en la tarea de *vetting* de TOIs sobre curvas de luz de TESS, bajo restricciones realistas de *dataset* y hardware. Sobre 1 576 TICs etiquetados (CP+FP) y una RTX 3050 de 4 GB de VRAM, Mamba *single-view locked* alcanzó AUC-ROC 0,763 en *test* sellado, superando al CNN *single-branch* inspirado en AstroNet (0,604) por +15,9 puntos porcentuales bajo condiciones idénticas. El *ensemble* de cinco semillas elevó la métrica a 0,806 con varianza significativamente menor que las predicciones individuales. El análisis de errores y la interpretabilidad multi-método coinciden en que los modos de falla restantes corresponden a curvas de baja SNR, no a errores sistemáticos del modelo. Como contribución metodológica adicional, el reporte propone un protocolo *multi-seed* como sustituto justificado de *K-fold* para arquitecturas costosas en hardware limitado.

Referencias

- [1] Gu, A. & Dao, T. (2023). *Mamba: Linear-Time Sequence Modeling with Selective State Spaces*. arXiv:2312.00752.
- [2] Shallue, C. J. & Vanderburg, A. (2018). *Identifying Exoplanets with Deep Learning*. AJ 155(2), 94.
- [3] Valizadegan, H. et al. (2022). *ExoMiner*. ApJ 926(2), 120.
- [4] Lightkurve Collaboration (2018). ascl:1812.013.
- [5] Ricker, G. R., Winn, J. N., Vanderspek, R., et al. (2015). *Transiting Exoplanet Survey Satellite (TESS)*. Journal of Astronomical Telescopes, Instruments, and Systems, 1(1), 014003.
- [6] NASA Exoplanet Archive. (2026). *TESS Objects of Interest (TOI) Table Data Column Definitions*. California Institute of Technology / IPAC.
- [7] Mikulski Archive for Space Telescopes (MAST). (2026). *TESS Archive Documentation: Data Product Overview*. Space Telescope Science Institute.
- [8] NASA. (2026). *TESS: Transiting Exoplanet Survey Satellite*. NASA Science Mission Directorate.